

FIL FRESHER TRAINING · HANDS-ON

Docker, step by step

From “it works on my machine” to a multi-container Python app — one command at a time.

10 steps

Python + Flask app

Docker Engine + Compose v2

YOUR LEARNING PATH

Ten small steps, each one builds on the last

01

Install & verify

02

Pull images

03

Run containers

04

Inspect & debug

05

Write a Dockerfile

06

**Build & run your
app**

07

**Keep data in
volumes**

08

**Connect with
networks**

09

Docker Compose

10

Clean up

WHY DOCKER?

“But it works on my machine...”

Without Docker

- Python 3.9 on your laptop, 3.12 on the server
- pip installs slightly different library versions
- A 20-step setup document nobody follows
- New joiners lose day one to environment setup

With Docker

- App + Python + libraries shipped as one image
- The same image runs on laptop, CI and server
- Setup becomes one command: docker run
- Broke something? Delete it and start fresh in seconds

WHAT IS DOCKER?

Docker packages your app and everything it needs into a **container — a lightweight, isolated process that runs the same everywhere.**

Standard box

Like a shipping container: one shape, whatever is inside.

Runs anywhere

Any machine with Docker can run it — laptop, CI, cloud.

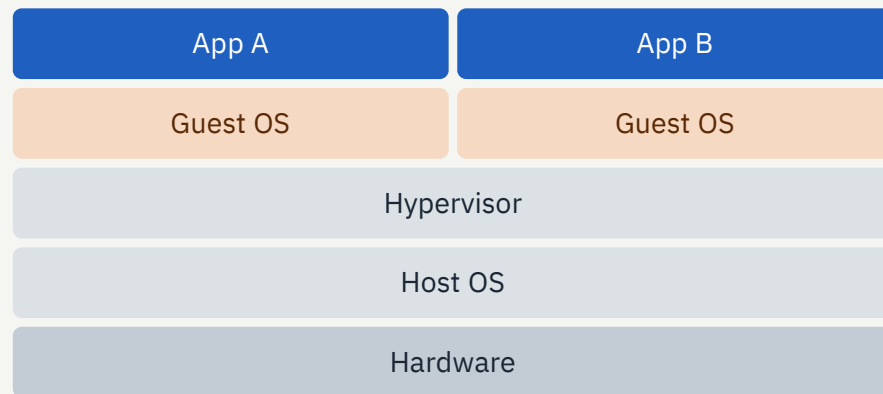
Isolated

Its files, processes and ports don't clash with other apps.

CONTAINERS VS VIRTUAL MACHINES

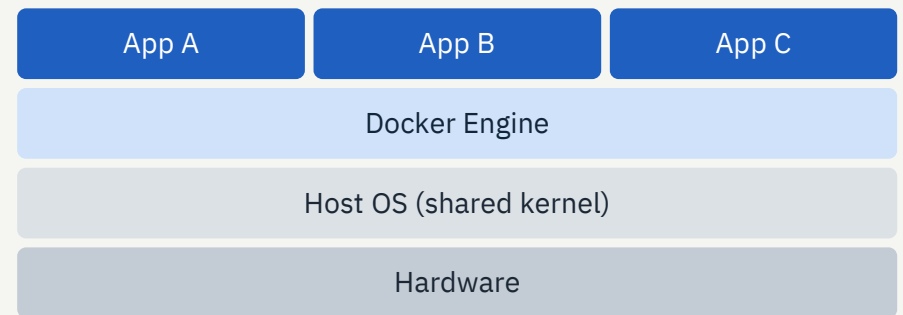
Containers share the host kernel, so they are light

Virtual machine



Each VM boots a full OS: GBs in size, minutes to start.

Container



No guest OS: MBs in size, starts in about a second.

FOUR WORDS TO KNOW

Image, container, Dockerfile, registry



Image

A read-only template: OS files + Python + your code.
Think **class**.



Container

A running instance of an image. Think **object** — many from one image.



Dockerfile

A text recipe of steps that builds an image.



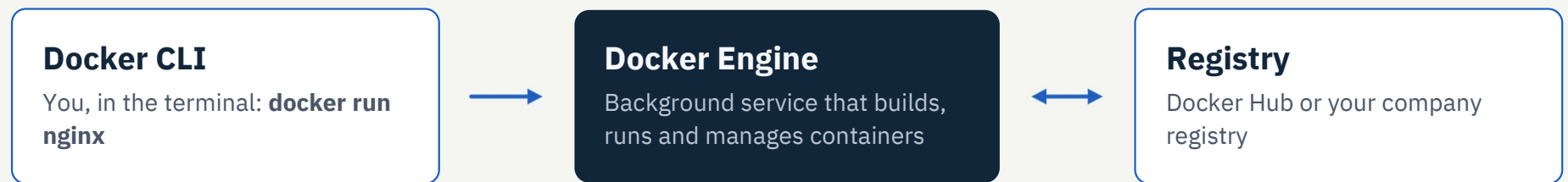
Registry

Where images are stored and shared, e.g. Docker Hub.

Dockerfile → [build](#) → Image → [run](#) → Container

HOW DOCKER WORKS

You type a command; the engine does the work



What happens on **docker run nginx**

1. The CLI sends the request to the Docker Engine.
2. The engine looks for the nginx image on your machine.
3. Not there? It pulls it from the registry.
4. It creates a container from the image and starts it.

STEP 1 · INSTALL & VERIFY

Get Docker running and say hello

Windows / macOS

Install Docker Desktop, start it, wait for “Engine running”.

Linux

Install Docker Engine and the Compose plugin from docs.docker.com.

`docker version`

Client and Engine versions — both must show

`docker info`

Containers, images and storage on this machine

`docker run hello-world`

Pulls a tiny test image and runs it

You're done when you see “Hello from Docker!” printed in your terminal.

STEP 2 · IMAGES

Pull, list and remove images

```
docker pull python:3.13-slim
```

Download an image from Docker Hub

```
docker images
```

List local images (same as docker image ls)

```
docker rmi <image_id>
```

Remove an image (no container may use it)

```
docker build -t <name:tag> .
```

Build your own image — coming in Step 6

Reading an image name

python = repository, **3.13-slim** = tag (version + variant).

No tag?

Docker assumes **:latest**, which changes over time. Pin tags in real projects.

STEP 3 · RUN CONTAINERS

docker run: four ways you'll use it

```
docker run nginx
```

Start in the foreground (Ctrl+C stops it)

```
docker run -it python:3.13-slim bash
```

Interactive shell inside a container

```
docker run -d -p 8080:80 nginx
```

In the background, host 8080 → container 80

```
docker run -d --name web -p 8080:80 nginx
```

Same, with a name you can reuse

-i keep input open

-t give a terminal

-d detached (background)

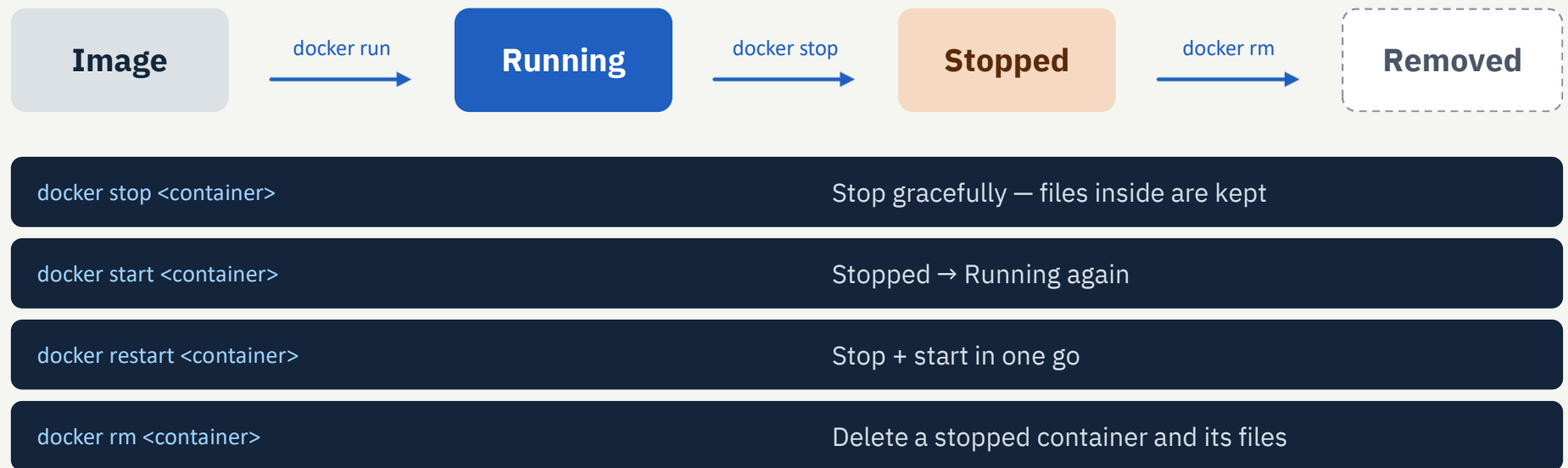
-p host:container port

--name a friendly label

Try it: open <http://localhost:8080> — the nginx welcome page means it worked.

STEP 3 · CONTAINER LIFECYCLE

A container is born, stops, restarts and is removed



STEP 4 · INSPECT & DEBUG

See what's running and look inside it

```
docker ps
```

List running containers

```
docker ps -a
```

All containers, including stopped ones

```
docker logs -f <container>
```

What the app printed; -f keeps following

```
docker exec -it <container> bash
```

Open a shell in a running container

```
docker inspect <container>
```

Full details: ports, mounts, env, IP

Shortcut: use the --name you gave it, or just the first 3–4 characters of the container ID.

STEP 5 · WRITE A DOCKERFILE

Package a small Python API

app.py

```
from flask import Flask
app = Flask(__name__)

@app.get("/")
def hello():
    return {"message": "Hello from Docker!"}

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Dockerfile

```
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

requirements.txt

```
flask>=3.1
```

STEP 5 · DOCKERFILE LINE BY LINE

Each instruction adds one cached layer

Instruction	What it does
FROM python:3.13-slim	Start from the official slim Python base image
WORKDIR /app	Use /app as the working folder inside the image
COPY requirements.txt .	Copy only the dependency list first
RUN pip install ...	Install libraries — a layer that is reused if unchanged
COPY . .	Copy the rest of your source code
EXPOSE 5000	Document the port the app listens on
CMD ["python", "app.py"]	Default command when a container starts

Why copy requirements.txt first? Code changes often, dependencies rarely — so a rebuild after a code edit skips the slow pip install.

STEP 6 · BUILD & RUN YOUR APP

From folder to running API in four commands

```
docker build -t hello-api:1.0 .
```

Build an image from this folder

```
docker run -d -p 8000:5000 --name api hello-api:1.0
```

Start it in the background

```
curl http://localhost:8000
```

Expect {"message": "Hello from Docker!"}

```
docker logs api
```

See Flask's request log

That final dot

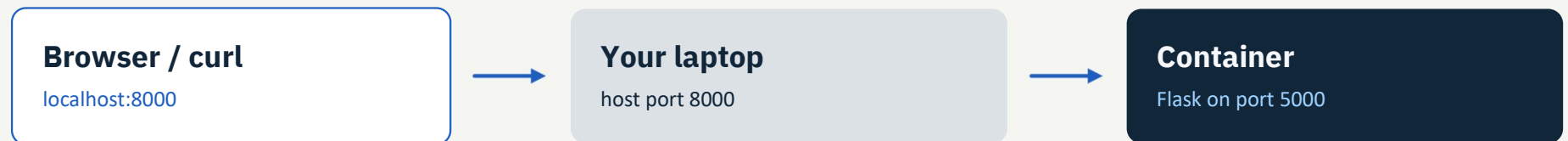
The `.` is the build context: the folder Docker sends to the engine.

Changed the code?

Rebuild with a new tag (1.1), then remove and re-run the container.

STEP 6 · PORT MAPPING

-p HOST:CONTAINER opens a door into the container



-p 8000:5000 → left = your machine, right = inside the container

“Port is already allocated”

Another container uses that host port. Pick a different left side.

“Connection refused”

The app listens on 127.0.0.1. Bind it to 0.0.0.0 inside the container.

STEP 7 · VOLUMES

Keep your data outside the container

Data written inside a container is lost on **docker rm** — so mount storage from outside.

Named volume — for app data

```
docker run -d --name cache \  
-v appdata:/data redis:8
```

Docker manages where it lives. Survives container removal.

Bind mount — for your code

```
docker run -it \  
-v "$(pwd)":/app python:3.13-slim bash
```

Maps your folder in; edits show up live inside.

```
docker volume ls
```

List volumes

```
docker volume rm <volume_name>
```

Delete a volume — and its data

Meet the app we will package: the Expense API

Project folder

```
expense-api/  
├─ app/  
│   ├── main.py    # FastAPI routes  
│   └── schemas.py # Pydantic models  
├─ db.py           # SQLite storage  
├─ tests/test_api.py  
├─ requirements.txt  
├─ requirements-dev.txt  
├─ .dockerignore  
└─ Dockerfile      ← you write this
```

Endpoints

GET	/health	liveness check
POST	/expenses	add an expense
GET	/expenses	list, ?category=food
GET	/expenses/{id}	one expense or 404
DELETE	/expenses/{id}	remove one
GET	/expenses/summary	totals per category

Run it on your laptop first — if it fails here, Docker will not fix it

`pip install -r requirements-dev.txt → pytest -q → uvicorn app.main:app --reload → /docs`

Six moves from an empty file to a finished image

01

Base image

FROM python:3.13-slim

02

Settings & folder

ENV · WORKDIR /app

03

Dependencies

COPY requirements · RUN pip

04

Non-root user

RUN useradd appuser

05

Code, drop root

COPY app/ · USER appuser

06

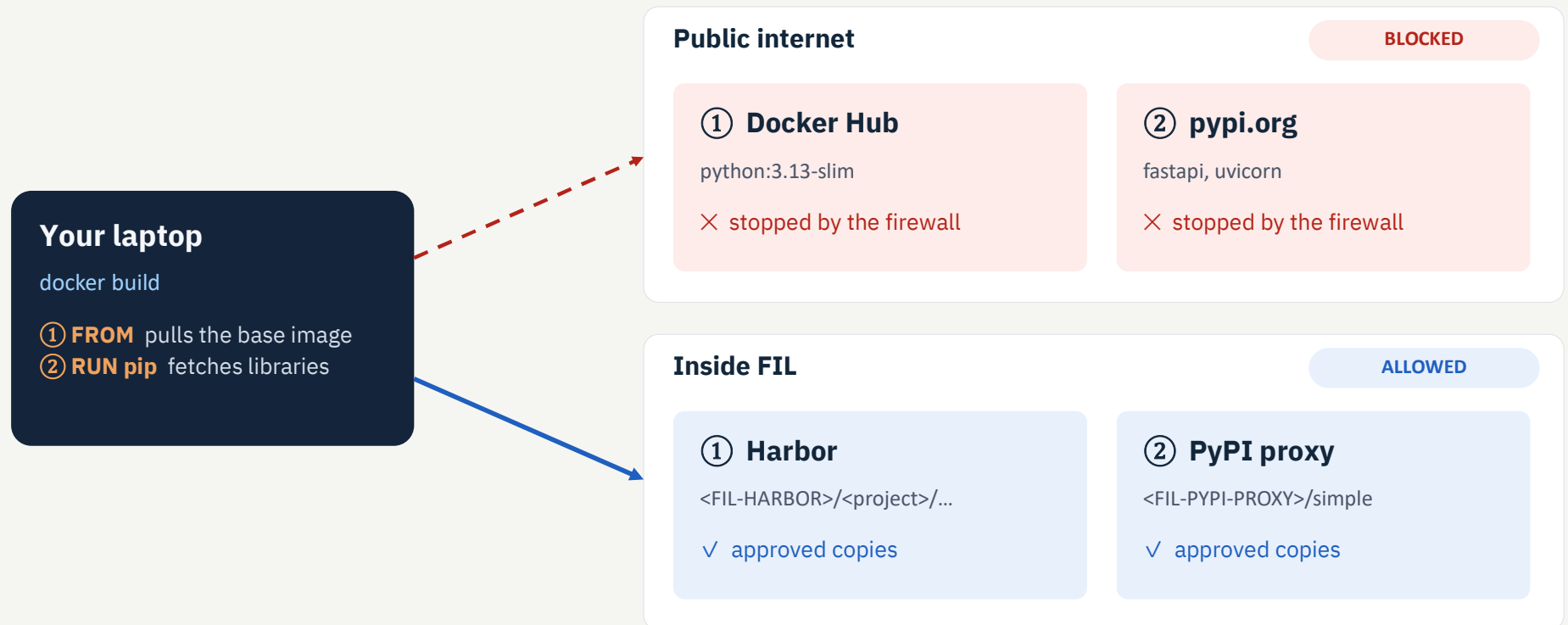
Port & start

EXPOSE · HEALTHCHECK · CMD

After every move: build with a step tag, then run one check

`docker build $FIL_ARGS -t expense-api:step2 .` then `docker run --rm expense-api:step2 <check>`

Inside FIL, the build must take a different road



Same Dockerfile, different source: two build arguments tell it where to fetch from (next slide).

Two build arguments point the build at FIL's copies

In the Dockerfile

```
ARG BASE_IMAGE=python:3.13-slim
FROM ${BASE_IMAGE}
...
ARG PIP_INDEX_URL=https://pypi.org/simple
RUN pip install --no-cache-dir -r requirements.txt
```

ARG = a value you pass at build time

The value after = is the default. --build-arg replaces it for one build. pip reads PIP_INDEX_URL by itself, so no pip command changes.

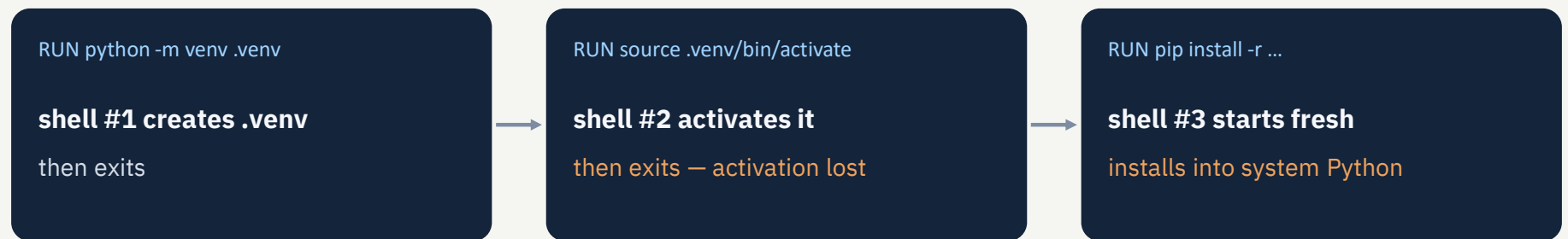
In your terminal — set once, then add \$FIL_ARGS to every build

```
# bash / Git Bash / WSL
FIL_ARGS="--build-arg BASE_IMAGE=<FIL-HARBOR>/<project>/python:3.13-slim
--build-arg PIP_INDEX_URL=https://<FIL-PYPI-PROXY>/simple"
```

```
# PowerShell
$FIL_ARGS = "--build-arg", "BASE_IMAGE=<FIL-HARBOR>/<project>/python:3.13-slim",
"--build-arg", "PIP_INDEX_URL=https://<FIL-PYPI-PROXY>/simple"
```

Then: `docker build $FIL_ARGS -t expense-api:step2` . **Harbor says unauthorized?** `docker login <FIL-HARBOR>`

Why we don't create a venv inside the image



What trainees saw: `/bin/sh: 1: source: not found` OR `ensurepip is not available`

The container already isolates the app

On a laptop, a venv keeps one project's libraries away from another's. An image holds exactly one app, so installing straight into its Python is safe and simpler.

Need one? Set PATH, don't activate

```
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
ENV lasts for every later line and at run time.
```

Start from a pinned base, then set where we work

Dockerfile so far

```
ARG BASE_IMAGE=python:3.13-slim
FROM ${BASE_IMAGE}
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
WORKDIR /app
```

bright = added in this move, **grey** = already there

ARG BASE_IMAGE + FROM

Pinned python:3.13-slim by default; at FIL, \$FIL_ARGS swaps in the Harbor copy.

ENV and WORKDIR

No .pyc files, logs printed straight away (no buffering).
WORKDIR creates /app and cds into it.

Checkpoint

```
docker build $FIL_ARGS -t expense-api:step2 .
Builds in seconds — nothing installed yet
```

```
docker run --rm expense-api:step2 pwd
Prints /app
```

Install the libraries before copying the code

Dockerfile so far

```
ARG BASE_IMAGE=python:3.13-slim
FROM ${BASE_IMAGE}
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
WORKDIR /app

ARG PIP_INDEX_URL=https://pypi.org/simple
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

bright = added in this move, **grey** = already there

Why copy requirements.txt alone?

Docker reuses a layer until its inputs change. Code changes daily, libraries rarely, so pip only reruns when requirements.txt changes.

ARG PIP_INDEX_URL

pip reads this variable, so at FIL every download goes through the PyPI proxy.

Checkpoint

```
docker build $FIL_ARGS -t expense-api:step3 .
Watch pip download fastapi and uvicorn
```

```
docker run --rm expense-api:step3 pip show fastapi
Name: fastapi · Version: 0.143.0
```

Create a non-root user, copy the code, then drop root

Dockerfile so far

```
ARG BASE_IMAGE=python:3.13-slim
FROM ${BASE_IMAGE}
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
WORKDIR /app

ARG PIP_INDEX_URL=https://pypi.org/simple
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

RUN useradd --create-home appuser \
    && mkdir -p /app/data \
    && chown appuser /app/data

COPY app/ ./app/
USER appuser
ENV EXPENSE_DB_PATH=/app/data/expenses.db
```

bright = added in this move, **grey** = already there

Why not run as root?

If someone breaks into the app, they get a normal user, not root inside the container.

/app/data for SQLite

Owned by appuser and named in EXPENSE_DB_PATH, so a volume can hold the database later.

Checkpoint

```
docker build $FIL_ARGS -t expense-api:step5 .
```

Only the new layers run; the pip layer is CACHED

```
docker run --rm expense-api:step5 whoami
```

Prints appuser

```
docker run --rm expense-api:step5 ls app
__init__.py db.py main.py schemas.py
```

The finished Dockerfile

Dockerfile · [expense-api/](#)

```
ARG BASE_IMAGE=python:3.13-slim
FROM ${BASE_IMAGE}
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
WORKDIR /app
ARG PIP_INDEX_URL=https://pypi.org/simple
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
RUN useradd --create-home appuser \
    && mkdir -p /app/data \
    && chown appuser /app/data
COPY app/ ./app/
USER appuser
ENV EXPENSE_DB_PATH=/app/data/expenses.db
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

EXPOSE 8000

Documentation only. -p 8000:8000 opens the port.

HEALTHCHECK

docker ps shows (healthy) once /health answers.

CMD in JSON form

Uvicorn gets Ctrl+C and docker stop directly.

Build the image and read what Docker tells you

```
docker build $FIL_ARGS -t expense-api:1.0 .
```

Build the finished image

```
docker images expense-api
```

Every tag you built: step2 ... 1.0

```
docker history expense-api:1.0
```

One row per layer, newest on top

Rebuild as 1.1 after editing main.py

```
=> [1/6] FROM <FIL-HARBOR>/<project>/python:3.13-slim
=> CACHED [2/6] WORKDIR /app
=> CACHED [3/6] COPY requirements.txt .
=> CACHED [4/6] RUN pip install --no-cache-dir -r ...
=> CACHED [5/6] RUN useradd --create-home appuser ...
=> [6/6] COPY app/ ./app/
=> naming to docker.io/library/expense-api:1.1
```

Edited main.py?

Only [6/6] COPY app/ runs again.

Edited requirements.txt?

pip runs again, and every step below it.

LAB · RUN & TEST

Run it, call it, and prove the data survives

1 · Start it

```
docker run -d --name expense -p 8000:8000 -v expense-data:/app/data expense-api:1.0
```

2 · Check it

```
docker ps
STATUS: Up ... (healthy) after ~5 seconds
```

```
curl http://localhost:8000/health
{"status":"ok"}
```

```
curl http://localhost:8000/expenses/summary
{"count":1,"total":850.0,"by_category":{"food":850.0}}
```

```
docker logs expense
Uvicorn's access log, one line per request
```

3 · Add an expense

```
curl -X POST http://localhost:8000/expenses \
-H "Content-Type: application/json" \
-d '{"title":"Team lunch",
    "amount":850,"category":"food"}
```

Or: open localhost:8000/docs → POST /expenses → Try it out

4 · Prove the data survives

docker rm -f expense, run step 1 again, then call /expenses/summary.

Eight errors you are likely to hit, and the fix for each

failed to resolve / pull access denied

Docker Hub is blocked. Build with \$FIL_ARGS (Harbor).
Unauthorized? docker login <FIL-HARBOR>.

pip: timeout / Could not find a version

pypi.org is blocked. \$FIL_ARGS must set PIP_INDEX_URL to the FIL proxy.

SSL: CERTIFICATE_VERIFY_FAILED

FIL's root certificate is missing. Ask IT for it; never switch the check off.

source: not found (venv)

Delete the venv lines. Each RUN is a new shell; the image is already isolated.

Could not import module "app.main"

Wrong folder, or COPY app/ missing. Run docker build inside expense-api/.

Connection refused / empty reply

Uvicorn is listening on 127.0.0.1. CMD must pass --host 0.0.0.0.

unable to open database file

/app/data is not owned by appuser. Check the chown in Move 4.

__pycache__ inside the image

Use **/__pycache__ in .dockerignore; a bare name matches the top folder only.

Shaded = caused by the enterprise setup · white = mistakes in the Dockerfile or app

ENTERPRISE CHECKLIST

Eight things to check before you build inside FIL

1 • Harbor base image

Docker Hub is blocked. Use the approved Harbor copy, pinned to a version.

2 • docker login

Run it once if the Harbor project is private. The password never goes in a file.

3 • PyPI proxy

Set PIP_INDEX_URL. Laptop pip settings do not reach inside a build.

4 • Proxy & certificates

Docker Desktop › Settings › Resources › Proxies. TLS errors: ask IT for the CA.

5 • No secrets

ARG, ENV and COPY end up in the image. docker history shows build arguments.

6 • apt needs a mirror

OS packages need FIL's Debian mirror. This lab installs none.

7 • No venv

The container is the isolation. Each RUN is a new shell anyway.

8 • Push to Harbor

Tag with the Harbor address and push there, never to Docker Hub. Harbor may scan it.

HANDS-ON LAB · 40 MINUTES

Ship the Expense API as an image

1. Unzip the starter, run **pytest -q** and open **/docs** locally once.
2. Set **\$FIL_ARGS** once, write Moves 1–2, build **expense-api:step2**, run **pwd**.
3. Add Move 3, build **step3**, confirm with **pip show fastapi**.
4. Add Moves 4–5, build **step5**, confirm **whoami** prints appuser.
5. Add Move 6, build **expense-api:1.0** and read **docker history**.
6. Run it with **-p 8000:8000** and the **expense-data** volume; add three expenses.
7. Remove the container, run it again, show **/expenses/summary** still has them.
8. Bump the version in **main.py**, rebuild as **1.1** and spot the **CACHED** lines.

Stretch goal: start a second copy on port 8001 with the same volume. Add via one, read via the other. Why?

STEP 8 · NETWORKS

Containers talk to each other by name

```
docker network create appnet
```

```
docker run -d --name redis \
  --network appnet redis:8
```

```
docker run -d --name api \
  --network appnet -p 8000:5000 hello-api:1.0
```

network: appnet



The api connects to **redis:6379** — Docker's built-in DNS turns the name into an IP.

```
docker network ls
```

List networks

```
docker network inspect appnet
```

See which containers are attached

STEP 9 · DOCKER COMPOSE

Describe the whole app in one file

compose.yaml

```
services:
  api:
    build: .
    ports:
      - "8000:5000"
    depends_on:
      - redis
  redis:
    image: redis:8
    volumes:
      - redisdata:/data
volumes:
  redisdata:
```

services = containers

api is built from our Dockerfile; redis uses a ready image.

Same ideas as Steps 6–8

ports, volumes and names — written once instead of typed every time.

Network for free

Compose creates one network; api reaches redis by its service name.

STEP 9 · COMPOSE COMMANDS

One command starts the whole stack

Write `docker compose` (space), not `docker-compose`. The hyphen version is the retired Compose v1; v2 is built into Docker with the same sub-commands.

```
docker compose up -d --build
```

Build if needed, start all in background

```
docker compose ps
```

Status of each service

```
docker compose logs -f api
```

Follow one service's logs (omit api for all)

```
docker compose exec api bash
```

Shell inside a running service

```
docker compose restart
```

Restart services (no rebuild)

```
docker compose down
```

Stop and remove containers + network

STEP 9 · SCALING

Run several copies of a service

```
docker compose up -d --scale api=3
```

Three containers of the api service

The catch

With a fixed **"8000:5000"**, only one copy can own host port 8000. The others fail: “port is already allocated”.

Fix for practice

Give a port range so each copy gets its own:

```
ports:  
  - "8000-8002:5000"
```

In real systems a load balancer (e.g. nginx) sits in front and spreads traffic across the copies.

STEP 10 · CLEAN UP

Images and stopped containers eat disk — tidy up

`docker system df`

How much space Docker is using

`docker container prune`

Remove all stopped containers

`docker image prune`

Remove dangling (untagged) images

`docker system prune`

Stopped containers, unused networks, dangling images, build cache

Prune is permanent. Adding `-a` also removes every unused image; `--volumes` deletes volume data. Read the prompt before you type `y`.

GOOD HABITS

Six habits to build from day one

Pin versions

python:3.13-slim, not python:latest.

Start slim

Smaller images build faster and carry fewer vulnerabilities.

Add .dockerignore

Keep .git, .venv and __pycache__ out of the image.

One process each

api and redis are separate services, not one container.

No secrets inside

Pass passwords and keys at runtime, never in the Dockerfile.

Rebuild, don't patch

Change the Dockerfile and rebuild — never hand-edit a running container.

HANDS-ON LAB · 45 MINUTES

Ship hello-api end to end

1. Run **docker run hello-world** and read what it tells you.
2. Run nginx on port 8080, open it in a browser, then stop and remove it.
3. Create app.py, requirements.txt and the Dockerfile from Step 5.
4. Build hello-api:1.0, run it on port 8000 and call it with curl.
5. Change the message, rebuild as 1.1 and spot the CACHED layers.
6. Write compose.yaml with api + redis and run **docker compose up -d**.
7. Use **docker compose logs** and **exec** to look inside.
8. Tear down with **docker compose down**, then check **docker system df**.

Stretch goal: make the API count visits in redis and show that the count survives docker compose down and up.

CHEATSHEET RECAP

The commands you'll use every week

Images

```
docker pull python:3.13-slim
docker images
docker build -t name:tag .
docker rmi <image>
```

Containers

```
docker run -d -p 8080:80 --name web nginx
docker ps -a
docker logs -f web · docker exec -it web bash
docker stop / start / rm web
```

Data, network, cleanup

```
docker volume ls / rm <volume>
docker network ls / inspect <net>
docker system df
docker system prune
```

Compose

```
docker compose up -d --build
docker compose ps · logs -f
docker compose exec api bash
docker compose down
```

WRAP-UP

Questions?

You can now run, build, connect and compose containers. Next: using these images in CI pipelines.

Dockerfile → build → image → run → container